

NHẬP MÔN KHOA HỌC DỮ LIỆU

TRỰC QUAN HOÁ DỮ LIỆU

ThS. Vũ Hoài Thư



Nội dung

- 1 Đồ thị dạng đường
- 2 Đồ thị điểm rời rạc
- 3 Trục quan bằng thanh lỗi
- 4 Đồ thị đường viền
- 5 Histogram và mật độ
- 6 Đồ thị ba chiều
- 7 Dữ liệu địa lý

Trực quan hoá dữ liệu

- **Trực quan hóa dữ liệu** (Data visualization) là thể hiện dữ liệu thành các dạng đồ họa như đồ thị, biểu đồ hay sử dụng các phương pháp, công cụ khác nhau để minh họa dữ liệu được tốt nhất.
- Mục đích là biến nguồn dữ liệu thành thông tin được thể hiện một cách trực quan, dễ quan sát, dễ hiểu, truyền đạt rõ ràng những hiểu biết đầy đủ (insights) từ dữ liệu đến người xem, người đọc.



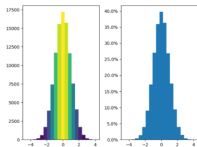
Trực quan hoá dữ liệu

Graphical excellence is that which gives to the viewer the greatest number of ideas in the shortest time with the least ink in the smallest space.

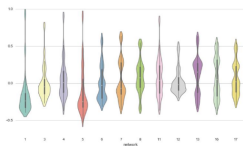
Quote from The Visual Display of Quantitative Information by Edward R. Tufte

Các công cụ

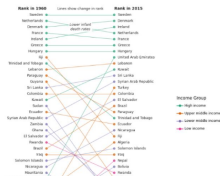
matplotlib



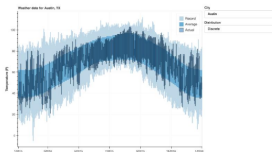
Seaborn



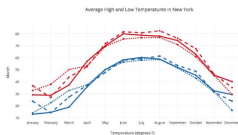
Plotnine (ggplot2)



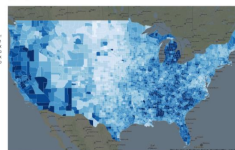
Bokeh



Plotly



geoplotlib



Trực quan hoá dữ liệu

Một số cách trình bày dữ liệu trực quan bằng các biểu đồ sau:

- Đồ thị dạng đường thẳng
- Đồ thị điểm rời rạc
- Trực quan hóa lỗi
- Đồ thị đường viền
- Histograms và mật độ
- Đồ thị ba chiều
- Dữ liệu địa lý

Thư viện trực quan hóa dữ liệu Matplotlib

- Matplotlib là thư viện trực quan hóa dữ liệu được xây dựng trên mảng NumPy và được thiết kế để hoạt động với ngăn xếp SciPy. Nó được John Hunter đưa ra vào năm 2002, cho phép vẽ sơ đồ kiểu MATLAB tương tác thông qua gnuplot từ dòng lệnh IPython.
- Một trong những tính năng quan trọng nhất của Matplotlib là khả năng hoạt động tốt với đa nền tảng nhiều hệ điều hành, điều này dẫn đến một số lượng lớn người dùng với sự phổ biến trong thế giới khoa học dữ liệu sử dụng.

Nạp thư viện matplotlib

Sử dụng viết tắt tiêu chuẩn khi nạp thư viện Matplotlib: plt bao gồm nhiều hàm sẽ được sử dụng thường xuyên trong suốt chương này.

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```


Đồ thị dạng đường

Đồ thị dạng đường

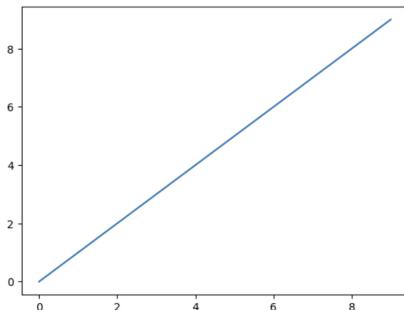
- Đồ thị dạng đường thẳng có lẽ là đồ thị đơn giản nhất trong tất cả các đồ thị.

```
import numpy as np
data = np.arange(10)
plt.plot(data)
```

✓ 0.2s

Python

[<matplotlib.lines.Line2D at 0x1675e5550>]



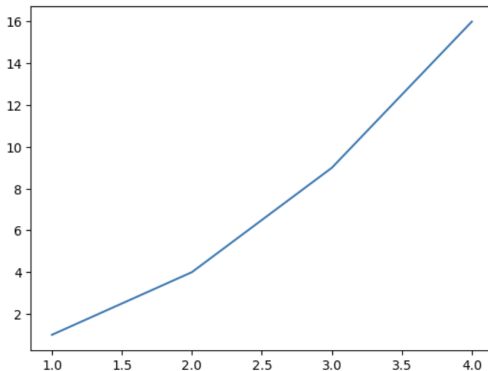
Đồ thị dạng đường

```
x = [1,2,3,4]
y = [1,4,9,16]
plt.plot(x,y)
plt.savefig('line_graph.png')
#plt.savefig('line_graph.png', dpi=400)
```



0.1s

Python



Đồ thị dạng đường

```
#Vẽ nhiều line graph
age = [25,26,27,28,29,30,31,32,33,34,35]
salary = [38496,42000,46752,49320,53200,56000,62316,64928,67317,68748,73752]
py_salary = [45372,48876,53850,57287,63016,65998,70003,70000,71496,75370,83640]

plt.plot(age,salary, "r-", marker=".") #marker: ký hiệu đánh dấu các điểm dữ liệu trong đồ thị
plt.plot(age,py_salary,"b--", marker=".")

#Thêm tiêu đề đồ thị, tiêu đề các trục
plt.title("Developer Salaries by Age")
plt.xlabel("Age")
plt.ylabel("Salary (USD)")
plt.grid(True) # thêm lưới cho dễ đọc giá trị
plt.legend(['All developers', 'Python developers']) #Thêm chú thích cho các đường
plt.show()
```

✓ 0.1s



Đồ thị dạng đường

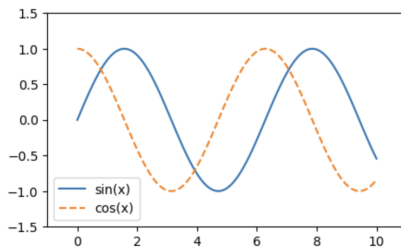
```
# Vẽ hàm sin và cos trên đồ thị
## Khởi tạo 100 điểm cách đều từ 0 đến 10
x = np.linspace(0, 10, 100)

## Khởi tạo figure
fig = plt.figure()

## Vẽ đồ thị sin(x) nét liền và cos(x) nét đứt
plt.plot(x, np.sin(x), "-")
plt.plot(x, np.cos(x), "--")

## Giới hạn trục X và trục Y
plt.xlim(-1, 11)
plt.ylim(-1.5, 1.5)
plt.legend(["sin(x)", "cos(x)"])
plt.show()
```

✓ 0.1s



Đồ thị dạng đường

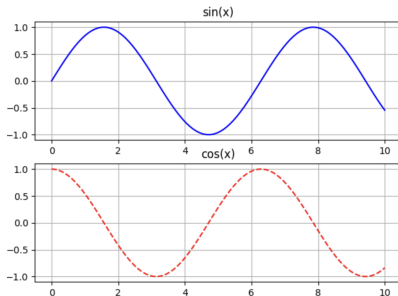
```
#Tạo nhiều biểu đồ con (subplot)
x = np.linspace(0, 10, 100)

#Tạo Figure có 2 hàng, 1 cột (2 subplot xếp dọc)
fig, ax = plt.subplots(2, 1, figsize=(7,5))

#Vẽ sin(x) ở subplot 1
ax[0].plot(x, np.sin(x), color="blue")
ax[0].set_title("sin(x)")
ax[0].grid(True)

#Vẽ cos(x) ở subplot 2
ax[1].plot(x, np.cos(x), color="red", linestyle="--")
ax[1].set_title("cos(x)")
ax[1].grid(True)
plt.show()
```

✓ 0.1s



Đồ thị dạng đường

```
#Liệt kê tên và mã màu trong Matplotlib
for name, hex in mpl.colors.cnames.items():
    print(name, hex, " ", end=" ")
```

✓ 0.0s

```
aliceblue #F0F8FF antiquewhite #FAEBD7 aqua
#00FFFF aquamarine #7FFFD4 azure #00FFFF
beige #F5F5DC bisque #FFE4C4 black #000000
blanchedalmond #FFEB3D blue #0000FF
blueviolet #8A2BE2 brown #A52A2A burlywood
#DEB887 cadetblue #5F9EA0 chartreuse
#7FFF00 chocolate #D2691E coral #FF7F50
cornflowerblue #6495ED cornsilk #FFF8DC
crimson #DC143C cyan #00FFFF darkblue
#00008B darkcyan #008B8B darkgoldenrod
#B8860B darkgray #A9A9A9 darkgreen #006400
darkgrey #A9A9A9 darkkhaki #BDB76B
darkmagenta .....
```

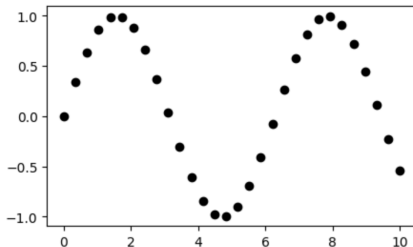
Đồ thị điểm rời rạc

Đồ thị điểm rời rạc

Đồ thị điểm rời rạc các điểm thay vì nối với nhau bằng đoạn thì được biểu diễn riêng lẻ bằng dấu chấm, hình tròn hoặc hình dạng khác bằng phương thức `plt.plot/ax.plot`

```
x = np.linspace(0, 10, 30)
y = np.sin(x)
plt.plot(x, y, 'o', color="black")
plt.show()
```

✓ 0.0s



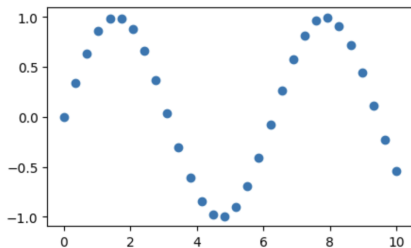
Đồ thị điểm rời rạc

Phương thức thứ hai, mạnh mẽ hơn để tạo đồ thị phân tán là hàm *plt.scatter*, được sử dụng rất giống với hàm *plt.plot*

```
plt.scatter(x, y, marker='o')
```

✓ 0.1s

<matplotlib.collections.PathCollection at 0x16a04e290>



Đồ thị điểm rời rạc

```
#Vẽ biểu đồ phân tán dạng bubble chart
##Tạo bộ sinh số ngẫu nhiên
rng = np.random.RandomState(0)

##Tạo 100 số ngẫu nhiên cho trục X
x = rng.randn(100)

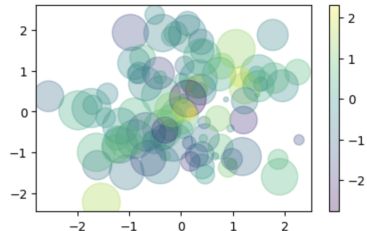
##Tạo 100 số ngẫu nhiên cho trục Y
y = rng.randn(100)

## Ánh xạ màu theo giá trị
color = rng.randn(100)

##Kích thước điểm ngẫu nhiên
sizes = 1000*rng.rand(100)

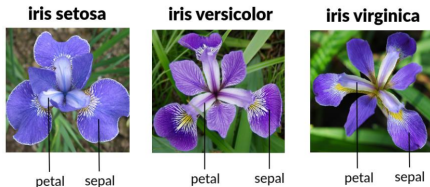
## Vẽ các điểm rải rác theo phân phối chuẩn
plt.scatter(x, y, c=color, s=sizes, alpha=0.3, cmap='viridis')
plt.colorbar()
```

✓ 0.1s



Đồ thị điểm rời rạc

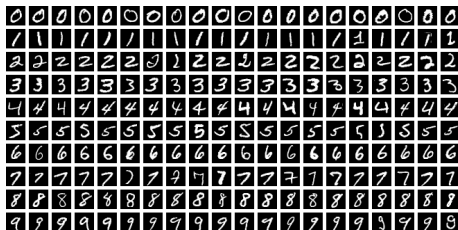
Ứng dụng cụ thể: trực quan hóa tập dữ liệu hoa Iris bằng scatter plot



Bộ dữ liệu bao gồm 50 mẫu từ mỗi loài trong số ba loài Iris (Iris setosa ,Iris virginica và Iris versicolor). Bốn đặc điểm được đo từ mỗi mẫu: chiều dài và chiều rộng của các lá đài và cánh hoa , tính bằng cm.

Đồ thị điểm rời rạc

Ứng dụng cụ thể: trực quan hóa tập dữ liệu MNIST bằng scatter plot
 Gợi ý: Giảm chiều bằng Isomap



Trực quan bằng thanh lỗi

Trực quan bằng thanh lỗi

- Đối với bất kỳ phép đo khoa học nào, việc tính toán chính xác các lỗi cũng quan trọng không kém việc báo cáo chính xác con số. Khi trực quan hóa dữ liệu và các kết quả, việc hiển thị các lỗi một cách hiệu quả có thể giúp đồ thị truyền tải thông tin đầy đủ hơn nhiều.
- Thanh lỗi (Error bar) được sử dụng làm đại diện cho sự biến đổi của một điểm dữ liệu. Điều này cung cấp về mức độ chính xác của điểm dữ liệu. Mức độ biến thiên càng nhiều thì điểm dữ liệu trong biểu đồ càng kém chính xác.

Trực quan bằng thanh lỗi

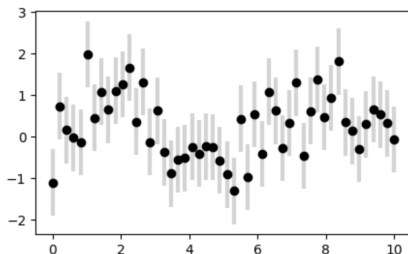
```
# Tạo 50 điểm từ 0 đến 10
x = np.linspace(0, 10, 50)

# Tạo biên độ sai số chuẩn
dy = 0.8

# Tạo dữ liệu: sin(x) + nhiễu ngẫu nhiên
y = np.sin(x) + dy * np.random.randn(50)

plt.errorbar(x, y, yerr=dy,          # thêm thanh sai số theo trục Y
             fmt='o',               # kiểu marker: vòng tròn
             color='black',          # màu của marker
             ecolor='lightgray',     # màu thanh sai số
             elinewidth=3,           # độ dày thanh sai số
             capsize=0);             # không vẽ mũ ở đầu thanh
```

✓ 0.1s



Đồ thị đường viền

Đồ thị đường viền

- Đôi khi sẽ hữu ích khi hiển thị dữ liệu ba chiều ở không gian hai chiều bằng cách sử dụng các đường viền hoặc vùng mã màu chuyển tiếp khác nhau.
- Có ba hàm Matplotlib có thể hữu ích cho nhiệm vụ này: `plt.contour` cho các ô đường viền, `plt.contourf` cho các ô đường viền được tô màu và `plt.imshow` để hiển thị hình ảnh.

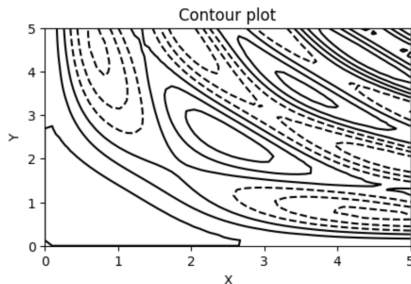
Đồ thị đường viền

```
# Tạo lưới
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 40)
X, Y = np.meshgrid(x, y)

# Tạo hàm 2 biến
Z = np.sin(X) * np.sin(Y) + np.cos(X*Y)

# Vẽ contour
plt.contour(X, Y, Z, colors='black')
plt.title("Contour plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

✓ 0.1s



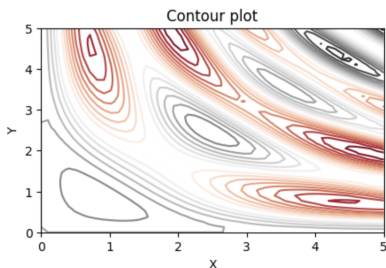
Đồ thị đường viền

```
# Tạo lưới
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 40)
X, Y = np.meshgrid(x, y)

# Tạo hàm 2 biến
Z = np.sin(X) * np.sin(Y) + np.cos(X*Y)

# Vẽ contour
## Chia giá trị Z thành 20 mức khác nhau, colormap Red-Gray (chuyển từ đỏ -> xám -> đen)
plt.contour(X, Y, Z, 20, cmap='RdGy')
plt.title("Contour plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

✓ 0.1s



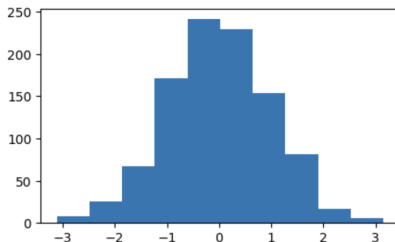
Histogram và mật độ

Histogram và mật độ

Histograms là một biểu đồ đơn giản để hiểu tập dữ liệu, đồ thị cơ bản được trực quan hoá bằng một dòng mã lệnh, sau khi nhập một tập dữ liệu như sau:

```
# Tạo 1000 số ngẫu nhiên theo phân phối chuẩn  $N(0,1)$   
data = np.random.randn(1000)  
  
# Vẽ histogram phân bố dữ liệu  
plt.hist(data)  
plt.show()
```

✓ 0.0s



Histogram và mật độ

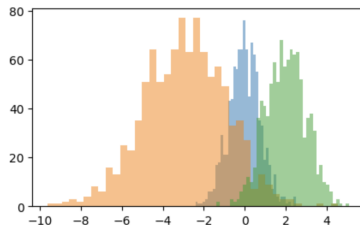
```
#Vẽ nhiều phân phối khác nhau
# Tạo ngẫu nhiên 1000 điểm dữ liệu theo phân phối chuẩn  $N(0, 0.8^2)$ 
x1 = np.random.normal(0, 0.8, 1000)

# Tạo ngẫu nhiên 1000 điểm dữ liệu theo phân phối chuẩn  $N(-3, 2^2)$ 
x2 = np.random.normal(-3, 2, 1000)

#Tạo ngẫu nhiên 1000 điểm dữ liệu theo phân phối chuẩn  $N(2, 1^2)$ 
x3 = np.random.normal(2, 1, 1000)

plt.hist(x1, alpha=0.5, bins=40)
plt.hist(x2, alpha=0.5, bins=40)
plt.hist(x3, alpha=0.5, bins=40)
plt.show()
```

✓ 0.1s



Đồ thị ba chiều

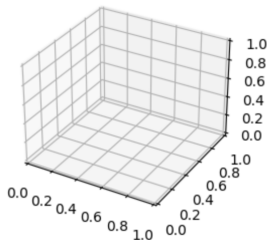
Đồ thị ba chiều

Matplotlib có một số gói vẽ sơ đồ ba chiều đã được xây dựng trên màn hình hai chiều. Để sử dụng biểu đồ ba chiều bằng cách nhập bộ công cụ *mplot3d*.

```
from mpl_toolkits import mplot3d
%matplotlib inline

fig = plt.figure()
#Tạo hệ trục 3D rỗng
ax = plt.axes(projection='3d')
```

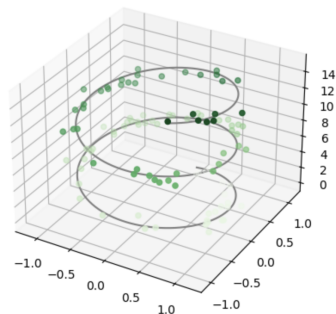
✓ 0.1s



Đồ thị ba chiều

```
#Khởi tạo trục 3D
ax = plt.axes(projection='3d')
#Khởi tạo trục z gồm 1000 điểm cách đều từ 0 đến 15
zline = np.linspace(0, 15, 1000)
#Trục x=sin(z)
xline = np.sin(zline)
#Trục y=cos(z)
yline = np.cos(zline)
# Vẽ đường xoắn ốc màu xám
ax.plot3D(xline, yline, zline, 'gray')
# Tạo z ngẫu nhiên trong [0, 15]
zdata = 15 * np.random.random(100)
# Tạo x = sin(z) + nhiễu
xdata = np.sin(zdata) + 0.1*np.random.randn(100)
# Tạo y = cos(z) + nhiễu
ydata = np.cos(zdata) + 0.1*np.random.randn(100)
# Vẽ đồ thị
ax.scatter3D(xdata, ydata, zdata, c=zdata, cmap='Greens')
```

✓ 0.1s



Dữ liệu địa lý

Dữ liệu địa lý

- Một loại trực quan hóa phổ biến trong khoa học dữ liệu là dữ liệu địa lý.
- Công cụ chính của Matplotlib cho kiểu trực quan hóa này là bộ công cụ Bản đồ cơ sở
- Cartopy là một thư viện Python mã nguồn mở, được xây dựng dựa trên Matplotlib và PROJ để hỗ trợ vẽ bản đồ và trực quan hóa dữ liệu địa lý.
- Cartopy được xem là người kế thừa của Basemap (đã ngừng phát triển từ 2020).
- Bên cạnh đó hiện nay API Google Maps được sử dụng phổ biến và là lựa chọn tốt hơn để trực quan hóa bản đồ chuyên sâu.

Dữ liệu địa lý

```
import cartopy.crs as ccrs
import matplotlib.pyplot as plt
# Khởi tạo Figure
plt.figure(figsize=(7, 5))

# Tạo bản đồ với phép chiếu Orthographic
ax = plt.axes(projection=ccrs.Orthographic(central_longitude=-100, central_latitude=50))

# Hiển thị ảnh nền
ax.stock_img()

# Thêm đường bờ biển
ax.coastlines()
plt.show()
```

✓ 0.2s



Bài tập nhóm chương 2+3

Tiến hành thu thập dữ liệu từ một trang các trang web sau (Chọn 1 trong các website bên dưới, không quá 2 nhóm/ 1 đề tài):

- <https://www.nhatot.com/mua-ban-bat-dong-san-ha-noi>
- <https://batdongsan.com.vn/nha-dat-ban-ha-noi>
- <https://moso.vn/>
- <https://alonthadat.com.vn/nha-dat/can-ban/nha-dat/1/ha-noi.html>
- <https://bonbanh.com/oto>
- <https://xe.chotot.com/mua-ban-oto-ha-noi>
- <https://oto.com.vn/>
- <https://tinbanxe.vn/oto/mua-ban/>
- <https://www.carmudi.vn/>

Bài tập nhóm chương 2+3

Yêu cầu:

- 1, Thu thập các thông tin cơ bản từ các website trên:
 - 1.1 Với các BDS: Cần thu thập được các thông tin cơ bản: Ngày đăng, Loại hình căn hộ, diện tích, giá, giấy tờ pháp lý, số phòng ngủ, số phòng vệ sinh, tình trạng nội thất).
 - 1.2 Với ô tô: Cần thu thập được các thông tin cơ bản: Ngày đăng bài, Năm SX, Xuất xứ, Địa điểm, Kiểu dáng, Số km đã đi, Hộp số, Tình trạng, Nhiên liệu
- 2, Hiển thị dữ liệu thu thập được, lưu lại trong file .CSV
- 3, Làm sạch dữ liệu (Xử lý giá trị thiếu, ngoại lai).
- 4, Co giãn chuẩn hóa dữ liệu.
- 5, Vẽ các biểu đồ trực quan với dữ liệu đã thu thập được (bất kỳ dạng biểu đồ nào có thể sử dụng) và đưa ra nhận xét.

Bài tập nhóm chương 2+3

Lưu ý:

1. Cần thu thập (crawl) tối thiểu được 2000 mẫu
2. Deadline: 25/10/2025
3. Mỗi nhóm cần nộp lại: Báo cáo kết quả + Code (Có thể up lên github và gắn link vào báo cáo kết quả).